COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structure Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	August 4, 2026	**DATE SUBMITTED:**	August 4, 2026
NAME:	Jaime D.C. Godino Jr.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 Queues

1. PROCEDURE OUTPUT

ILO A

main.cpp

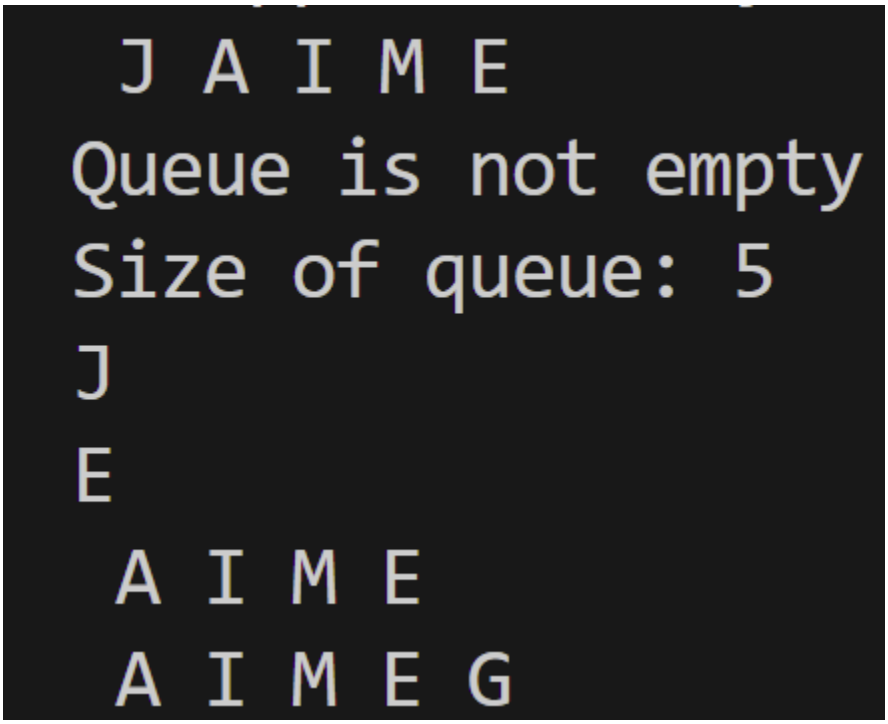
```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);


int main(){
    //create an object
    std::queue<char> myQ;

    myQ.push('J');
    myQ.push('A');
    myQ.push('I');
    myQ.push('M');
    myQ.push('E');
    display(myQ);
    //try all member functions:
    //empty
    if (myQ.empty()){
        std::cout << "Queue is empty" << std::endl;
    }else{
        std::cout << "Queue is not empty" << std::endl;
    }
    //size
    std::cout << "Size of queue: " << myQ.size() <<std::endl;
    //front
    std::cout <<myQ.front() << std::endl;
    //back
    std::cout <<myQ.back() << std::endl;
    //pop
    myQ.pop();
    //display
```

output



Observation/Remarks

The program displayed the queue contents, identified that the queue is not empty, and updated the elements after dequeue and enqueue operations. This basically shows the FIFO principle

ILO B

queueLL.h

```
#ifndef QUEUELL_H
#define QUEUELL_H


//create a class the node
template <typename T>
class qNode{
    public:
        T data;
        qNode<T>* next;
};

//create the new node
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

//enqueue operation
template<typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    //create the new node
    qNode<T>* newNode = new_node(newData);

    //inserting an item into a non empty queue
```

```
29      if((*frontPtr)==nullptr && (*backPtr)==nullptr){
30          (*frontPtr) = newNode;
31          (*backPtr) = newNode;
32      }
33      //inserting an itme into a non empty queue
34      (*backPtr)->next = newNode;
35      (*backPtr) = newNode;
36  }
37
38  template <typename T>
39  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
40      //create a temporary node
41      qNode<T>* deleteNode = nullptr; //temp variable
42      deleteNode = (*frontPtr);
43
44      //check if the queue has only one node
45      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr) {
46          (*frontPtr) = nullptr;
47          (*backPtr) = nullptr;
48          delete deleteNode;
49          return;
50      }
51      //delete the node
52      (*frontPtr) = (*frontPtr)->next;
53      deleteNode->next = nullptr;
```

```
54      delete deleteNode;
55  }
56
57  //check the front
58  template <typename T>
59  void checkFront(qNode<T>* frontPtr){
60      if(frontPtr == nullptr){
61          std::cout << "Queue is empty\n";
62          return;
63      }
64
65      std::cout << "Front: " << frontPtr->data << std::endl;
66  }
67  //check the back
68  template <typename T>
69  void checkBack(qNode<T>* backPtr) {
70      if(backPtr == nullptr){
71          std::cout << "Queue is empty\n";
72          return;
73      }
74
75      std::cout << "Back: " << backPtr->data << std::endl;
76  }
77  //display all
78  template <typename T>
```

```
80      if(frontPtr == nullptr) {
81          std::cout << "Queue is empty \n";
82          return;
83      }
84
85      qNode<T>* current = frontPtr;
86
87      std::cout << "Queue: ";
88
89      while(current != nullptr){
90          std::cout << current->data << " ";
91          current = current->next;
92      }
93      std::cout << std::endl;
94  }
95  #endif
```

main.cpp

```cpp
#include <iostream>
#include "queueLL.h"

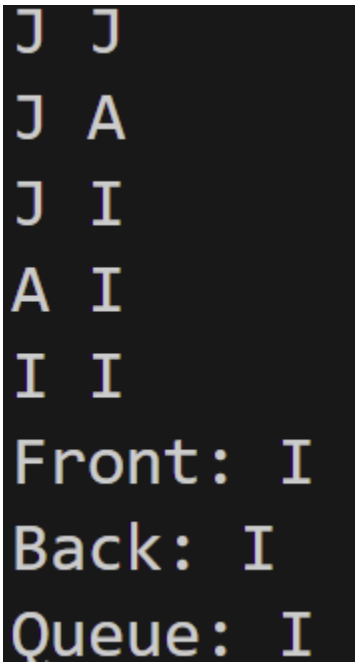

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('J', &front, &back);
    std::cout<< front->data << " " << back->data <<std::endl;
    enqueue('A', &front, &back);
    std::cout<< front->data << " " << back->data <<std::endl;
    enqueue('I', &front, &back);
    std::cout<< front->data << " " << back->data <<std::endl;

    dequeue(&front, &back);
    std::cout<< front->data << " " << back->data <<std::endl;
    dequeue(&front, &back);
    std::cout<< front->data << " " << back->data <<std::endl;

    checkFront(front);
    checkBack(back);
    displayAll(front);
    return 0;
}
```

Output



Observation/Remarks

This program implements the linked list of the queue. The font and back were updated properly during enqueue and dequeue operations, and the display function printed the queue elements in the correct order. Overall, the program performed the intended operations.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **NONE**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.